Государственное учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Военный факультет

Кафедра электронных вычислительных машин

ОТЧЕТ
по лабораторной работе № 3
«СОЗДАНИЕ РЕЛЯЦИОННОЙ СХЕМЫ ДАННЫХ»
Вариант: «Табачный завод»

Курсант Н.А.Шурхай

Преподаватель С.С. Силич

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Цель работы заключается в том, чтобы научиться использовать инструмент ERD Tool графического клиента pgAdmin для перевода ER-диаграммы в реляционную диаграмму. Это позволит получить наглядное представление структуры базы данных, включая таблицы, столбцы и их взаимосвязи, что является важным этапом в проектировании и реализации эффективных баз данных в PostgreSQL.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Установка PostgreSQL

Ниже поэтапно рассмотрен процесс установки СУБД PostgreSQL.

Открываю страницу загрузки, на которой представлены установочные файлы для различных операционных систем и их разрядности. Лучше всего использовать наиболее актуальную версию PostgreSQL. В настоящее время последней версией является 17.6, поэтому выбираю сборку 17.6 для Windows x86-64 и загружаю её (рисунок 2.2):

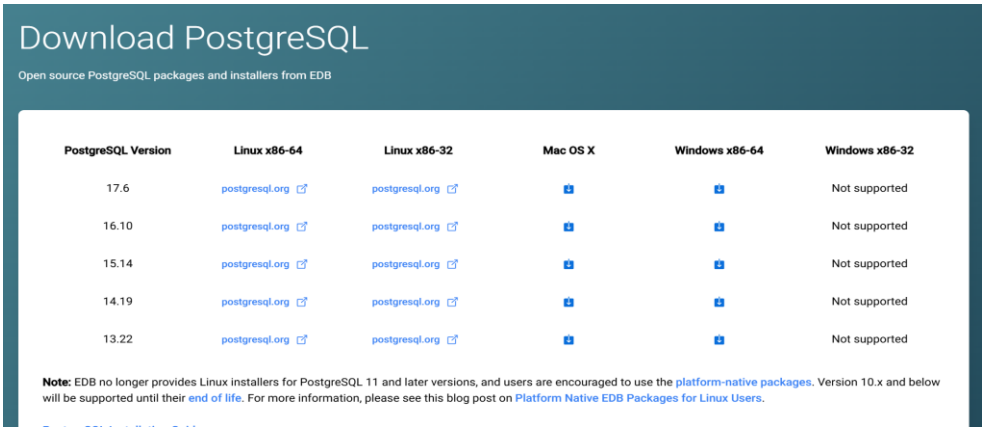


Рисунок 2.2 – Скачивание установочного файла

Когда загрузка завершится, установочный файл нужно запустить. После запуска открывается приветственное окно мастера установки (рисунок 2.3):

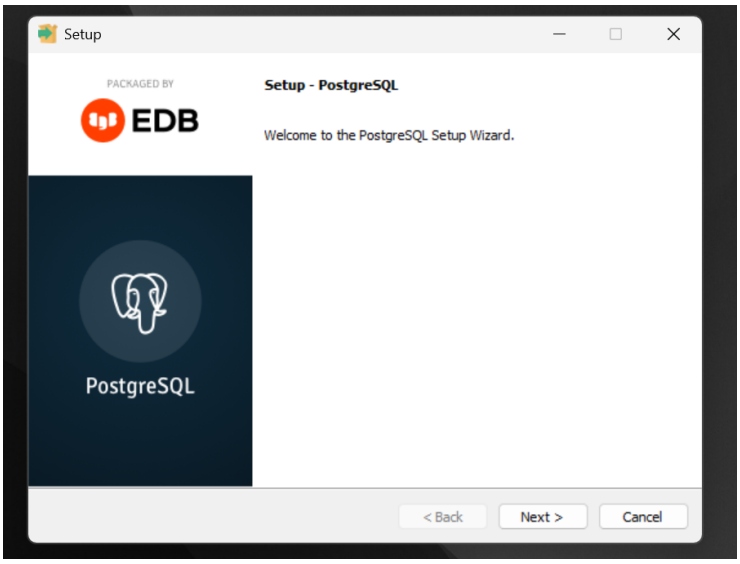


Рисунок 2.3 – Стартовое окно установщика

Задаю каталог, в который будет установлена PostgreSQL. Желательно оставить путь, предложенный установщиком по умолчанию, – это поможет

избежать возможных трудностей при дальнейшей работе с СУБД (рисунок 2.4):

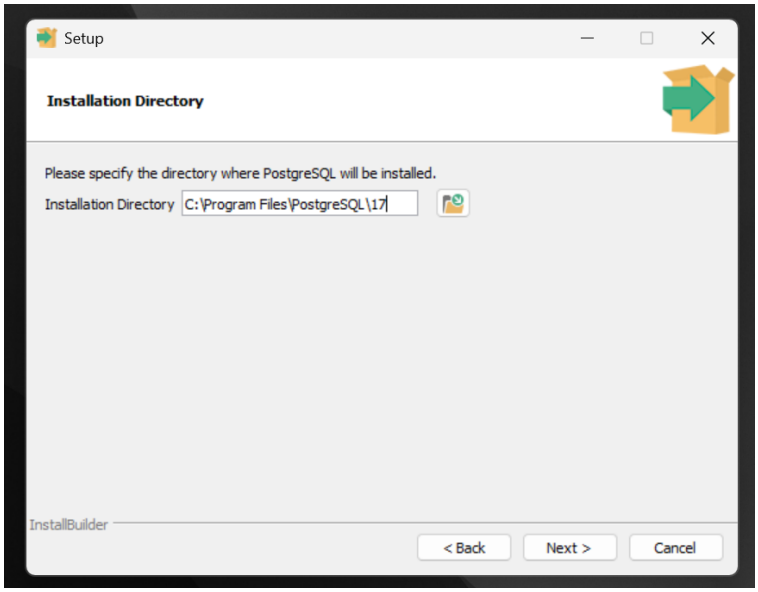


Рисунок 2.4 – Выбор каталога установки PostgreSQL

На следующем шаге выбираются компоненты для установки. Чтобы отметить компонент, нужно установить флажок рядом с его названием, а чтобы исключить лишний – снять его. Для выполнения работы достаточно компонентов PostgreSQL Server, pgAdmin 4 и Command Line Tools (рисунок 2.5):

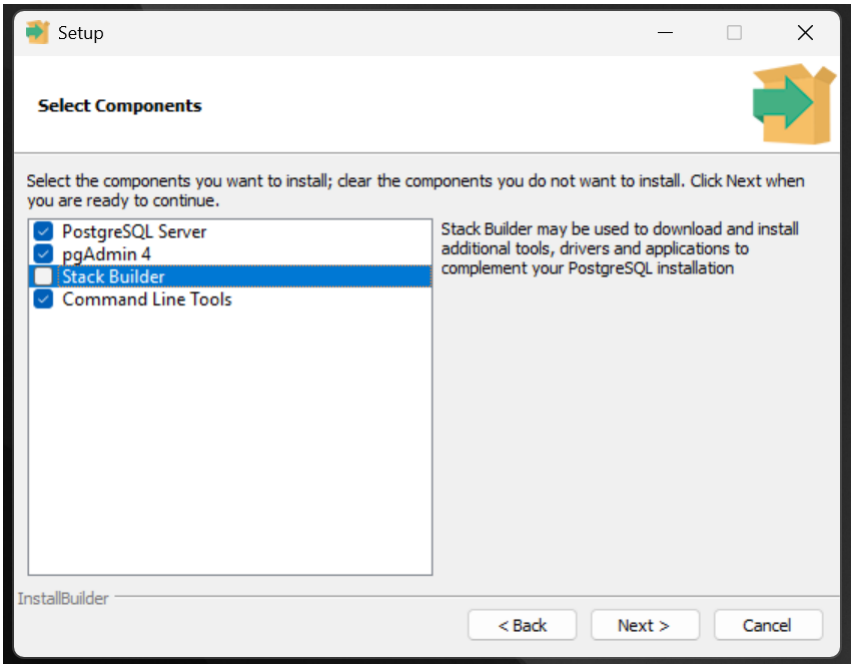


Рисунок 2.5 – Выбор устанавливаемых компонентов

Задаю расположение каталога data. В этом каталоге PostgreSQL хранит все сведения о базах данных, файлы конфигурации и журналы работы сервера (рисунок 2.6):

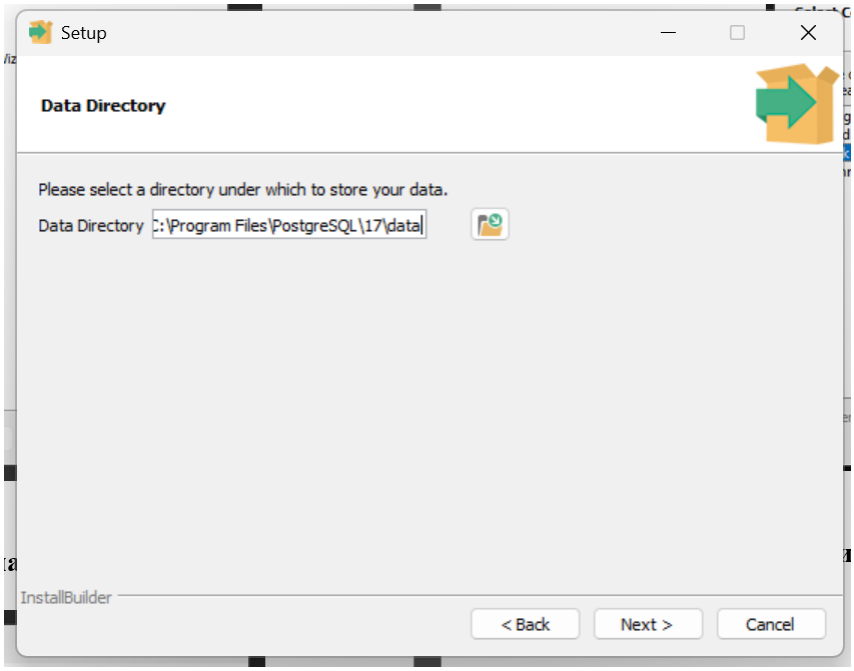


Рисунок 2.6 – Выбор расположения каталога data

Затем задаётся пароль для суперпользователя postgres. Его следует запомнить, поскольку именно он потребуется при подключении к базе данных (рисунок 2.7):

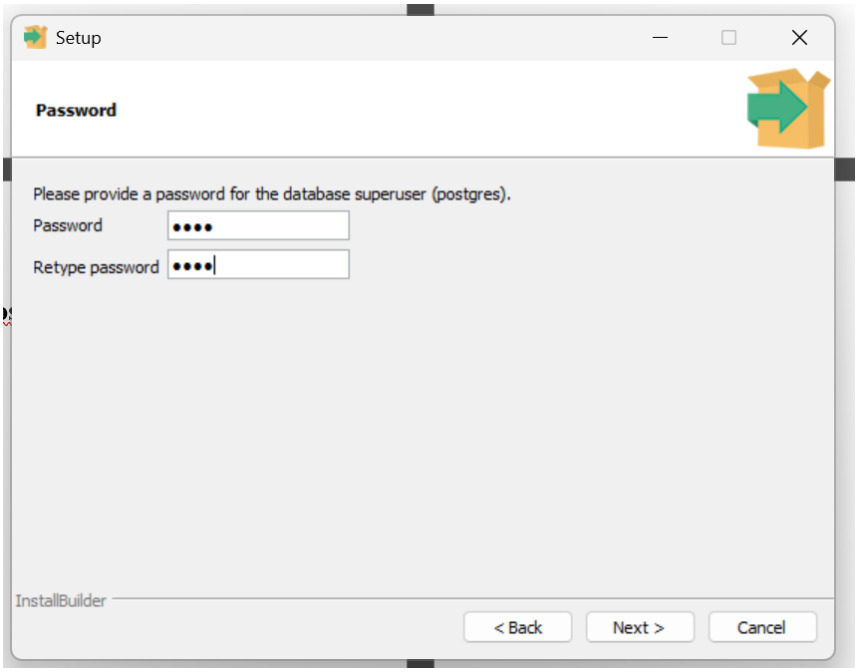


Рисунок 2.7 – Ввод пароля суперпользователя

После этого указывается порт, на котором сервер будет принимать подключения. Лучше оставить стандартное значение 5432, предложенное по умолчанию (рисунок 2.8):

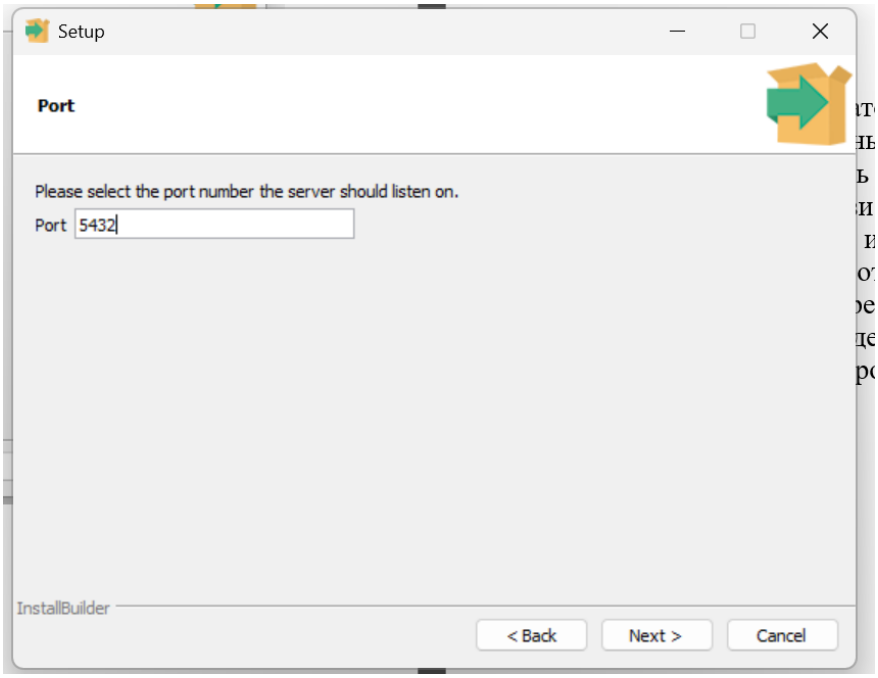


Рисунок 2.8 – Выбор порта

Выбираю локаль, которую будет использовать PostgreSQL. Этот параметр задаёт языковые и региональные настройки базы данных (рисунок 2.9):

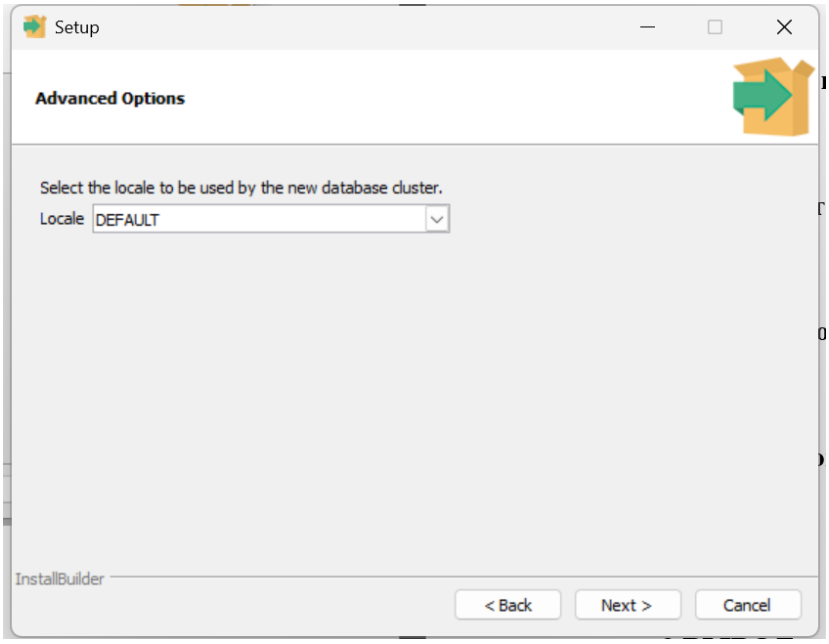


Рисунок 2.9 – Выбор локали

Затем мастер установки выводит перечень всех заданных параметров. Перед продолжением их стоит ещё раз внимательно проверить (рисунок 2.10):

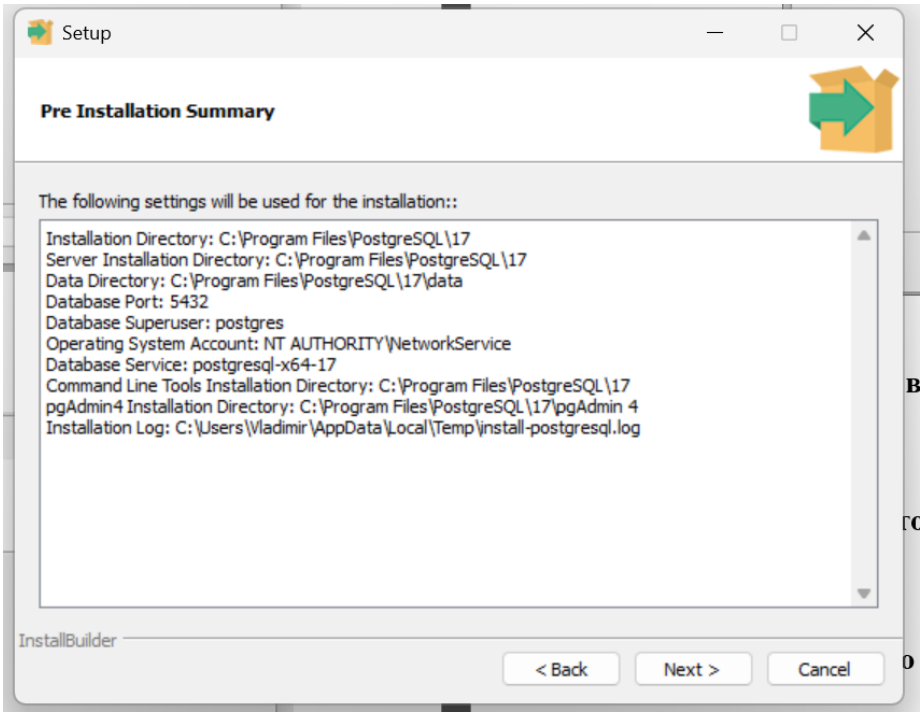


Рисунок 2.10 – Перечень параметров установки

Когда параметры проверены, установщик сообщает о готовности к началу установки (рисунок 2.11):

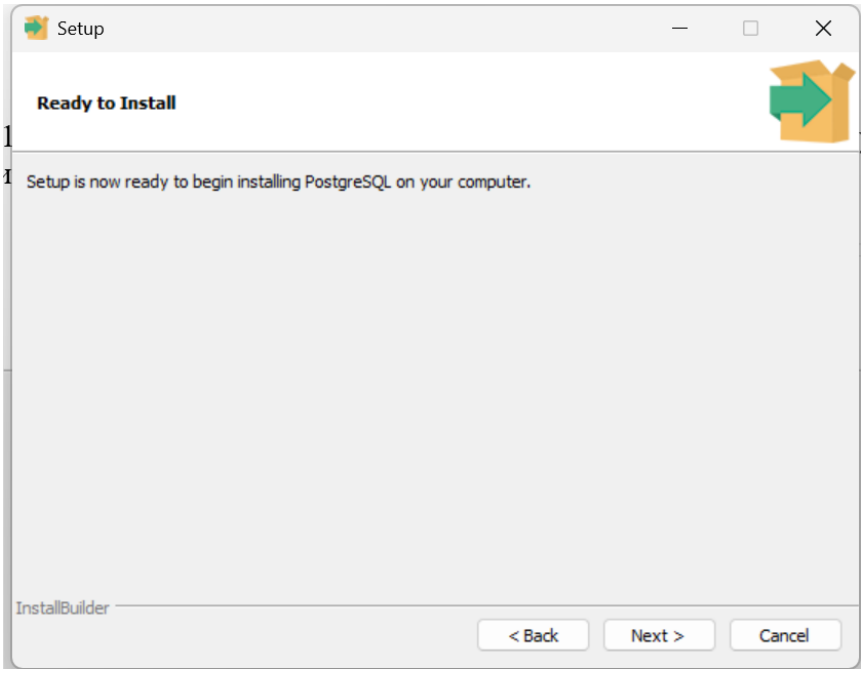


Рисунок 2.11 – Всё готово к установке

Запускаю процесс установки PostgreSQL (рисунок 2.12):

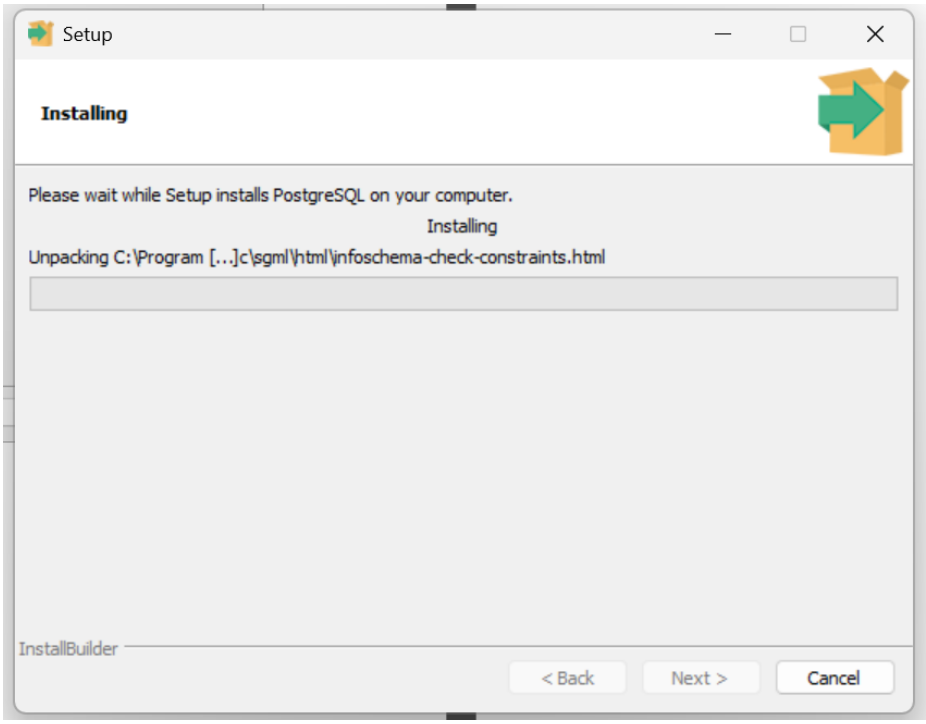


Рисунок 2.12 – Процесс установки

Установка завершилась успешно, PostgreSQL готова к работе (рисунок 2.13):

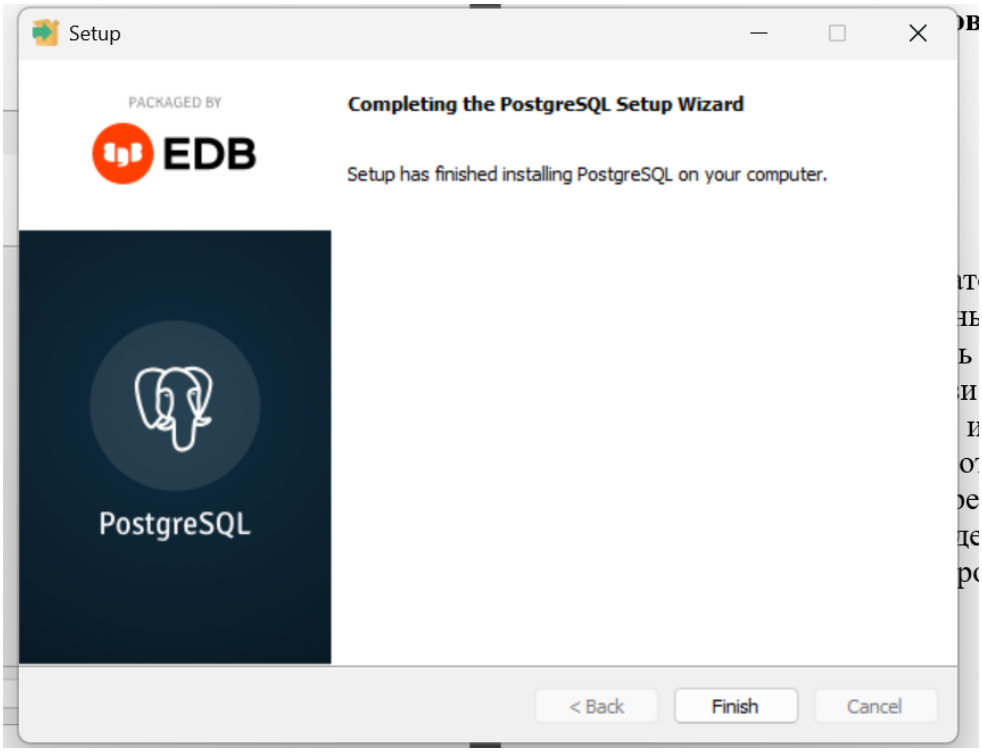


Рисунок 2.13 – Завершение установки

2.2 Использование ERD Tool

В качестве исходной используется следующая реляционная схема данных (рисунок 2.14):

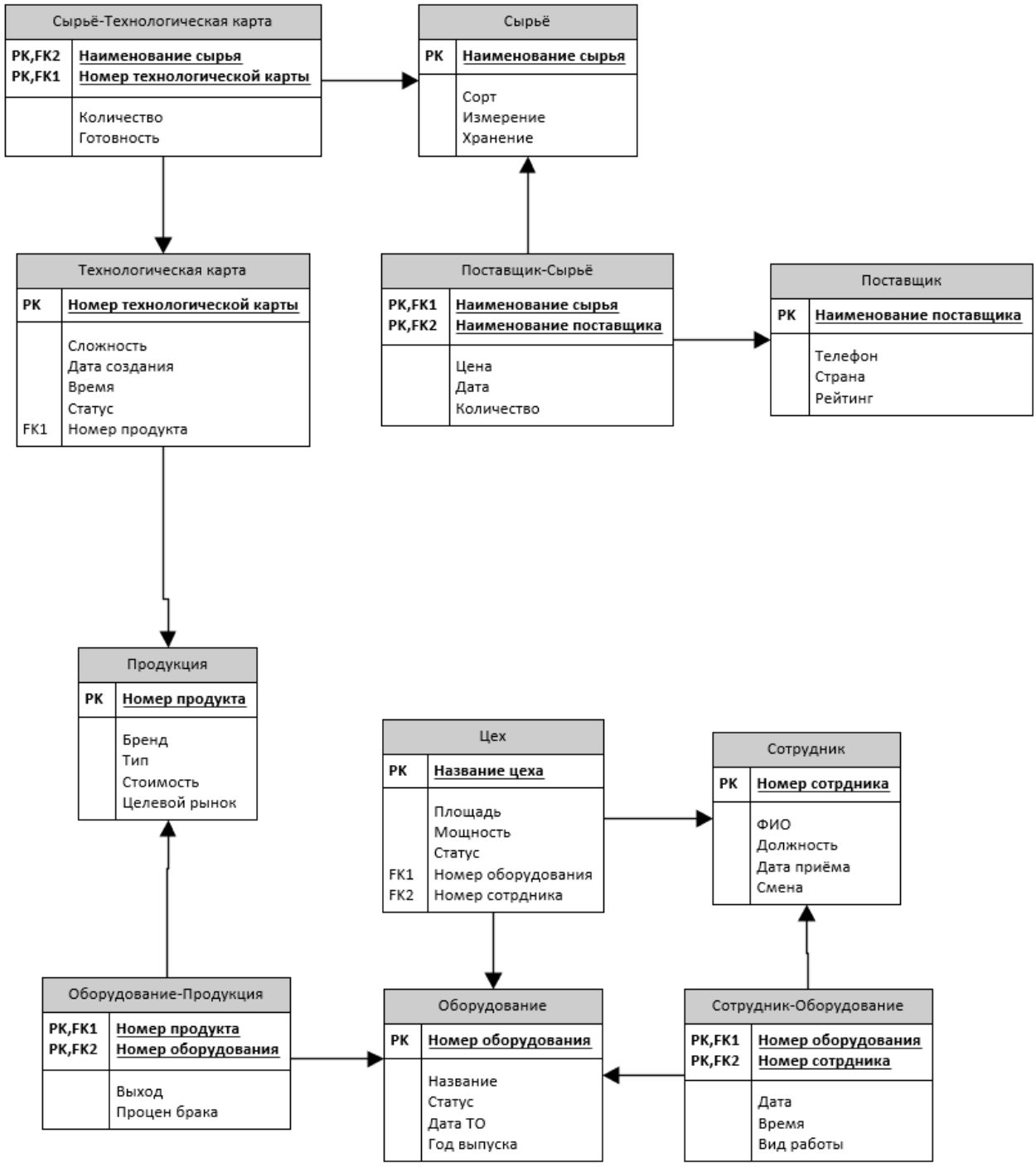


Рисунок 2.14 – Исходная реляционная схема данных

Для выполнения преобразования запускаю pgAdmin, затем в меню Tools выбираю пункт ERD Tool (рисунок 2.15):

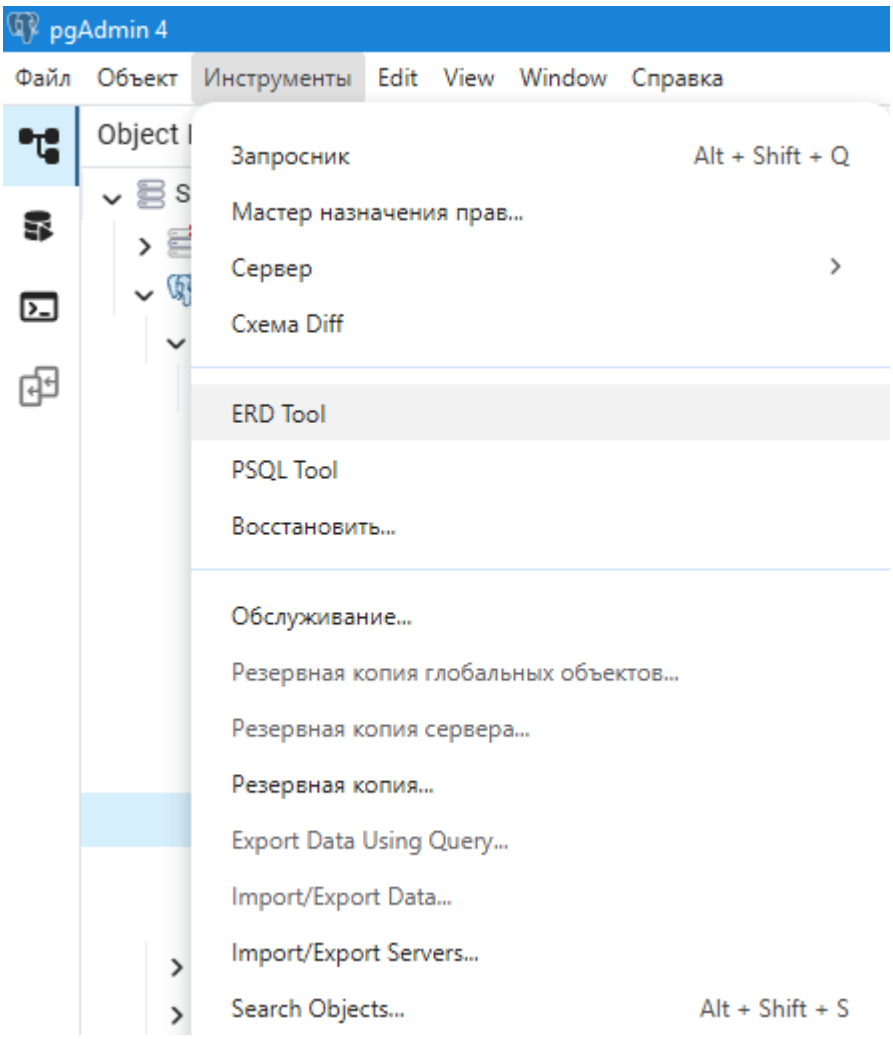


Рисунок 2.15 – Запуск ERD Tool

После запуска ERD Tool открывается рабочая область для построения таблиц. В ней будут располагаться таблицы, отражающие типы объектов ER-диаграммы, и связи между ними. Для добавления новой таблицы используется кнопка Add Table (рисунок 2.16):

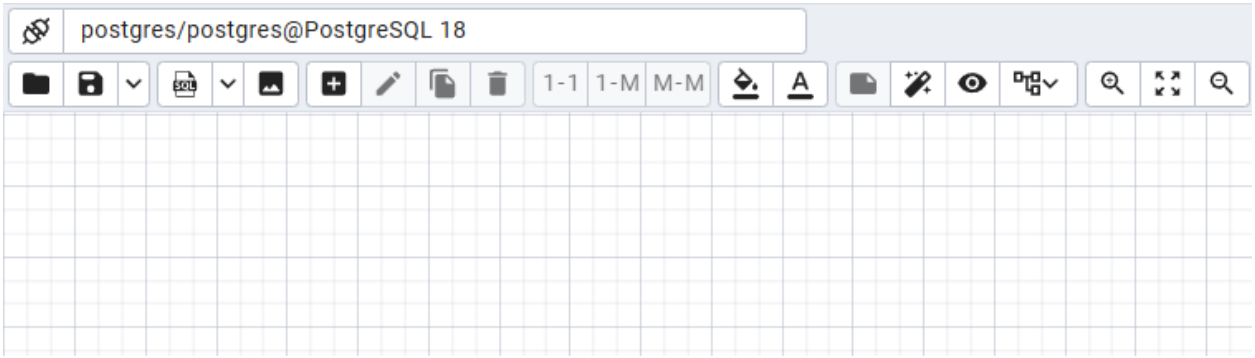


Рисунок 2.16 – Рабочая область ERD Tool

Затем появляется окно создания таблицы, где задаются её основные свойства: имя, столбцы, их типы данных, ограничения и прочие параметры. Заполняю основные свойства таблицы, результат показан на рисунке 2.17:

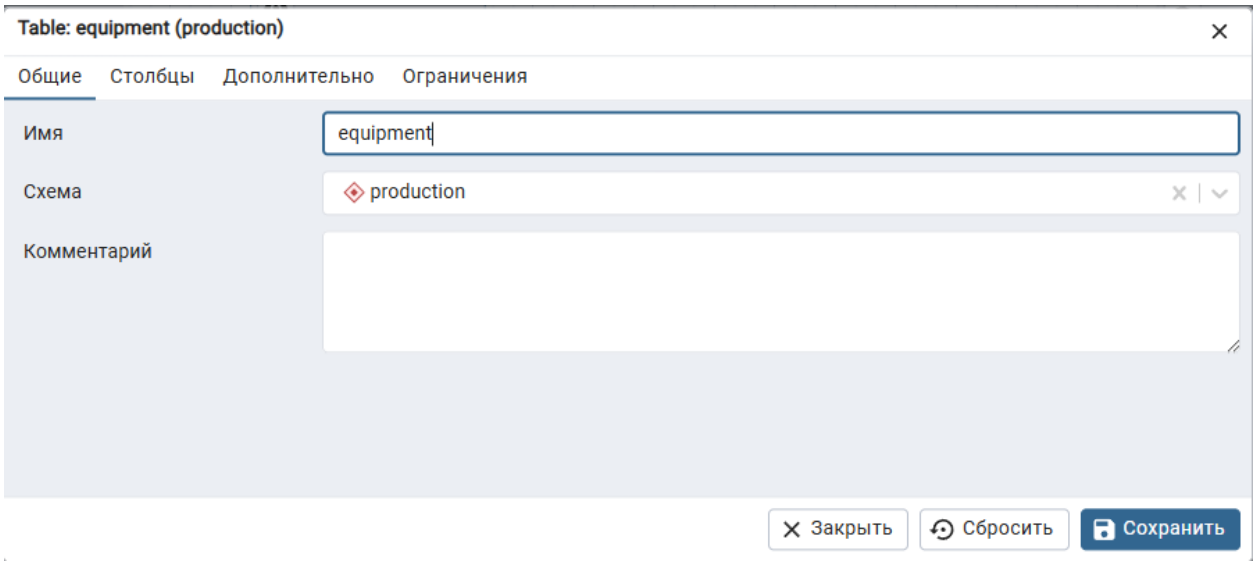


Рисунок 2.17 – Окно создания таблицы

Когда основные свойства заданы, нужно описать столбцы таблицы. Добавляю поля таблицы (рисунок 2.18):

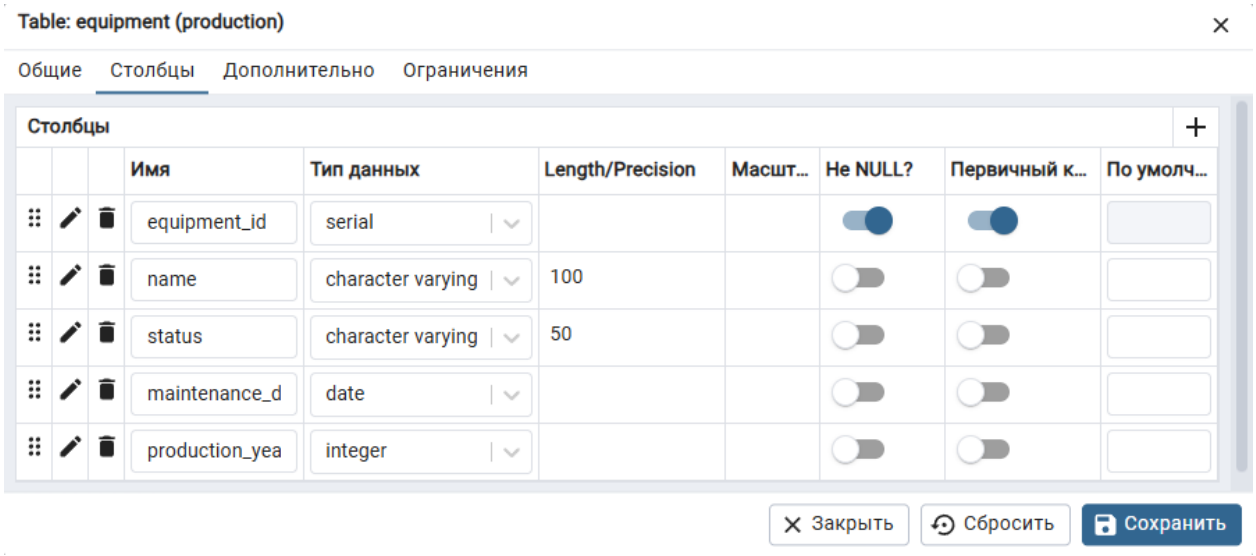


Рисунок 2.18 – Добавление столбцов таблицы

Связи типов «один-к-одному», «один-ко-многим» и «многие-ко-многим» можно реализовать, в частности, с помощью вкладки ограничений (Constraints). Чтобы добавить внешний ключ, нажимаю кнопку Add row (рисунок 2.19):

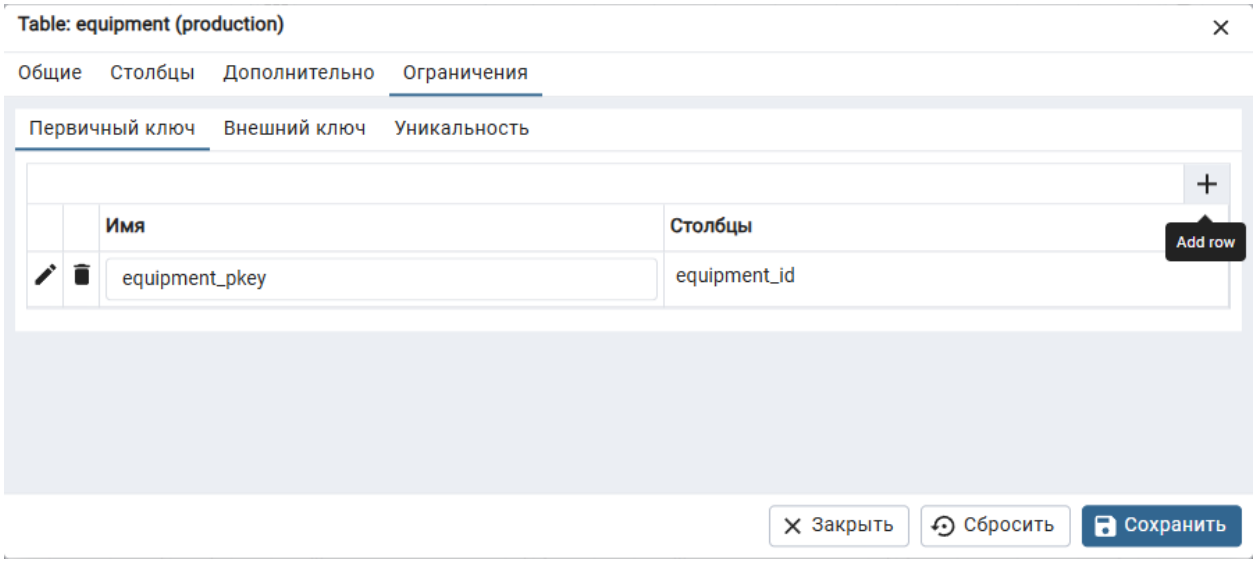


Рисунок 2.19 – Добавление внешнего ключа

После этого заполняются параметры внешнего ключа: столбец текущей таблицы, таблица, на которую он ссылается, и её поле. Указываю эти данные и нажимаю кнопку Add, чтобы добавить ключ (рисунок 2.20):

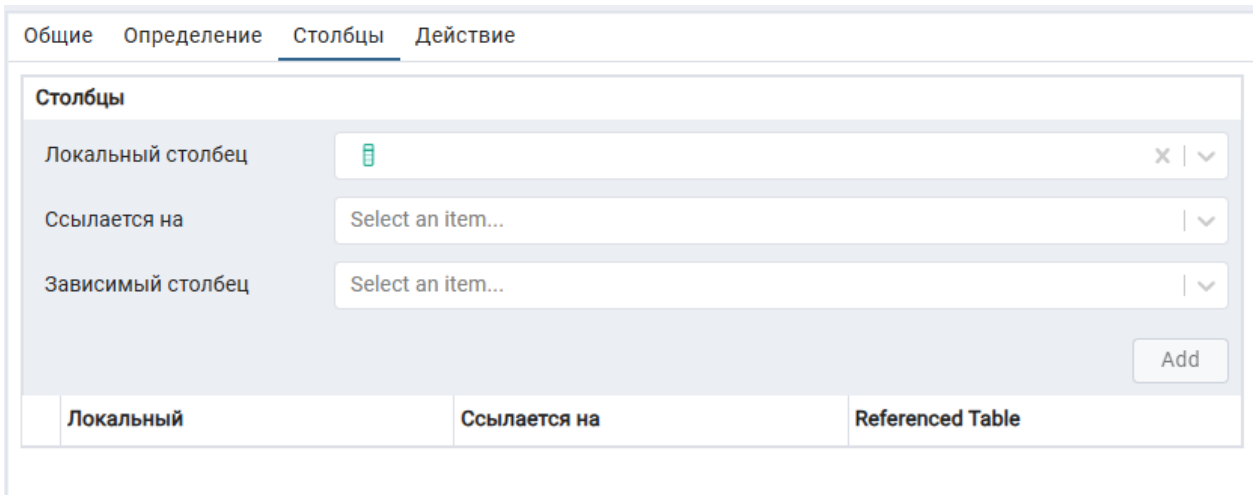


Рисунок 2.20 – Параметры внешнего ключа

Аналогичным способом создаются остальные таблицы, соответствующие типам объектов, и между ними устанавливаются определённые ранее связи. Итоговая реляционная модель для ER-диаграммы «Табачный завод» показана на рисунке 2.21:

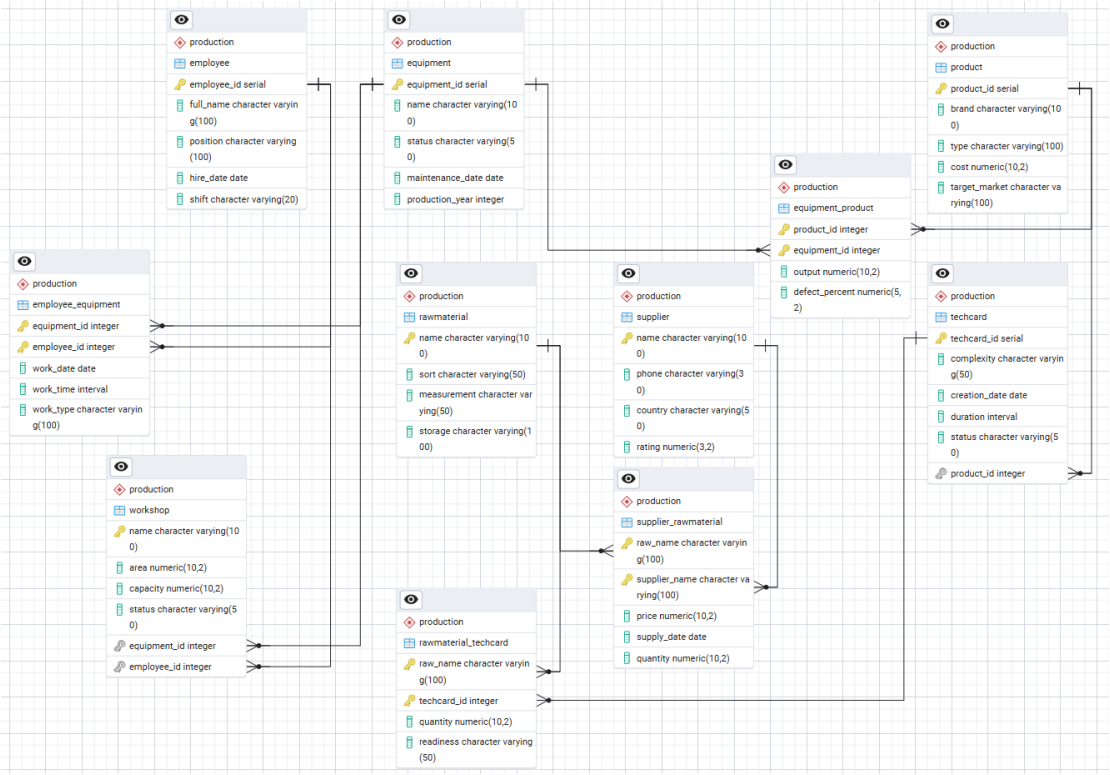


Рисунок 2.21 – Итоговая реляционная модель

На основе построенной реляционной модели можно автоматически получить SQL-код, создающий все её объекты. Для этого используется кнопка Generate SQL (рисунок 2.22):

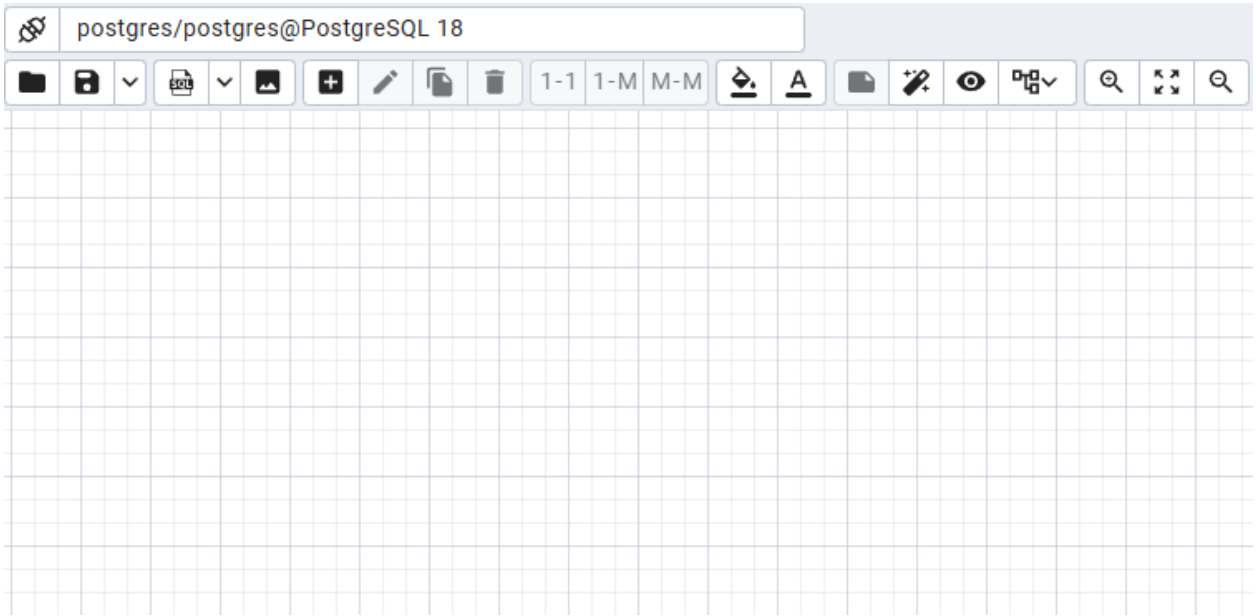


Рисунок 2.22 – Получение SQL-кода

Полученный код можно сразу выполнить. Для этого нажимаю кнопку

Execute script (рисунок 2.23):

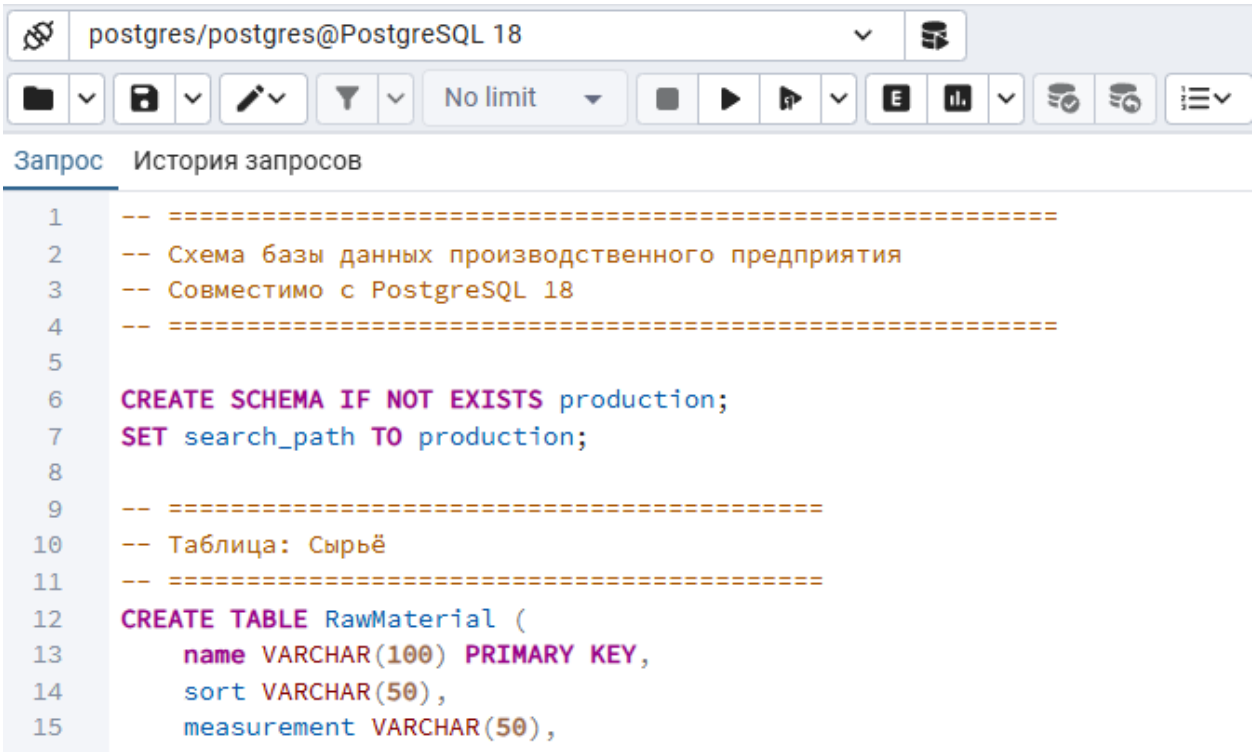


Рисунок 2.23 – Выполнение скрипта

Скрипт выполнился без ошибок, и в разделе Tables появились созданные таблицы и ключи (рисунок 2.24):

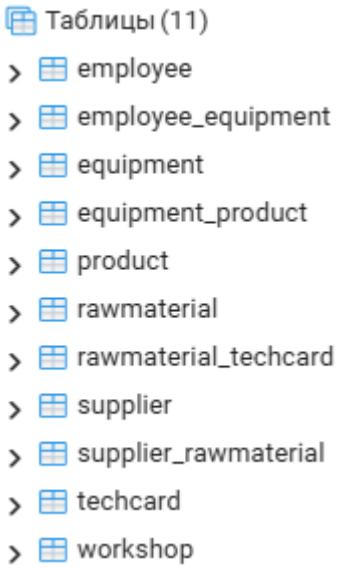


Рисунок 2.24 – Созданные таблицы в pgAdmin

Далее приведён SQL-код, сформированный на основе реляционной модели:

```
-- ============================================================
-- Схема базы данных производственного предприятия
-- Совместимо с PostgreSQL 18
-- ============================================================

CREATE SCHEMA IF NOT EXISTS production;
SET search_path TO production;

-- ========================================
-- Таблица: Сырьё
-- ========================================
CREATE TABLE RawMaterial (
    name VARCHAR(100) PRIMARY KEY,
    sort VARCHAR(50),
    measurement VARCHAR(50),
    storage VARCHAR(100)
);

-- ========================================
-- Таблица: Поставщик
-- ========================================
CREATE TABLE Supplier (
    name VARCHAR(100) PRIMARY KEY,
    phone VARCHAR(30),
    country VARCHAR(50),
    rating NUMERIC(3,2)
);

-- ========================================
-- Таблица: Поставщик-Сырьё (многие-ко-многим)
-- ========================================
CREATE TABLE Supplier_RawMaterial (
    raw_name VARCHAR(100),
    supplier_name VARCHAR(100),
    price NUMERIC(10,2),
    supply_date DATE,
    quantity NUMERIC(10,2),
    PRIMARY KEY (raw_name, supplier_name),
    FOREIGN KEY (raw_name) REFERENCES RawMaterial(name) ON
DELETE CASCADE,
    FOREIGN KEY (supplier_name) REFERENCES Supplier(name) ON
DELETE CASCADE
);

-- ========================================
-- Таблица: Продукция
-- ========================================
CREATE TABLE Product (
    product_id SERIAL PRIMARY KEY,
    brand VARCHAR(100),
```

```
    type VARCHAR(100),
    cost NUMERIC(10,2),
    target_market VARCHAR(100)
);

-- ========================================
-- Таблица: Технологическая карта
-- ========================================
CREATE TABLE TechCard (
    techcard_id SERIAL PRIMARY KEY,
    complexity VARCHAR(50),
    creation_date DATE,
    duration INTERVAL,
    status VARCHAR(50),
    product_id INT REFERENCES Product(product_id) ON DELETE
SET NULL
);

-- ========================================
-- Таблица: Сырьё-Технологическая карта (многие-ко-многим)
-- ========================================
CREATE TABLE RawMaterial_TechCard (
    raw_name VARCHAR(100),
    techcard_id INT,
    quantity NUMERIC(10,2),
    readiness VARCHAR(50),
    PRIMARY KEY (raw_name, techcard_id),
    FOREIGN KEY (raw_name) REFERENCES RawMaterial(name) ON
DELETE CASCADE,
    FOREIGN KEY (techcard_id) REFERENCES
TechCard(techcard_id) ON DELETE CASCADE
);

-- ========================================
-- Таблица: Оборудование
-- ========================================
CREATE TABLE Equipment (
    equipment_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    status VARCHAR(50),
    maintenance_date DATE,
    production_year INT
);

-- ========================================
-- Таблица: Оборудование-Продукция (многие-ко-многим)
-- ========================================
CREATE TABLE Equipment_Product (
    product_id INT,
    equipment_id INT,
    output NUMERIC(10,2),
    defect_percent NUMERIC(5,2),
```

```
        PRIMARY KEY (product_id, equipment_id),
        FOREIGN KEY (product_id) REFERENCES Product(product_id)
ON DELETE CASCADE,
        FOREIGN       KEY       (equipment_id)       REFERENCES
Equipment(equipment_id) ON DELETE CASCADE
    );

    -- ========================================
    -- Таблица: Сотрудник
    -- ========================================
    CREATE TABLE Employee (
        employee_id SERIAL PRIMARY KEY,
        full_name VARCHAR(100),
        position VARCHAR(100),
        hire_date DATE,
        shift VARCHAR(20)
    );

    -- ========================================
    -- Таблица: Сотрудник-Оборудование (многие-ко-многим)
    -- ========================================
    CREATE TABLE Employee_Equipment (
        equipment_id INT,
        employee_id INT,
        work_date DATE,
        work_time INTERVAL,
        work_type VARCHAR(100),
        PRIMARY KEY (equipment_id, employee_id),
        FOREIGN       KEY       (equipment_id)       REFERENCES
Equipment(equipment_id) ON DELETE CASCADE,
        FOREIGN       KEY       (employee_id)       REFERENCES
Employee(employee_id) ON DELETE CASCADE
    );

    -- ========================================
    -- Таблица: Цех
    -- ========================================
    CREATE TABLE Workshop (
        name VARCHAR(100) PRIMARY KEY,
        area NUMERIC(10,2),
        capacity NUMERIC(10,2),
        status VARCHAR(50),
        equipment_id INT,
        employee_id INT,
        FOREIGN       KEY       (equipment_id)       REFERENCES
Equipment(equipment_id) ON DELETE SET NULL,
        FOREIGN       KEY       (employee_id)       REFERENCES
Employee(employee_id) ON DELETE SET NULL
    );
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы ER-модель была преобразована в реляционную с помощью инструмента ERD Tool в pgAdmin. Использованный способ формирования первичных ключей в связующих таблицах дал возможность подробно разобраться в процессе создания таблиц и связей между ними.

С помощью ERD Tool были построены таблицы, соответствующие ER-диаграмме, заданы их свойства и настроены внешние ключи для связей различных типов. По готовой модели был автоматически сформирован SQL-скрипт, успешное выполнение которого подтвердило правильность построенной схемы.

Выполненная работа помогла закрепить навыки проектирования баз данных и работы с pgAdmin при администрировании PostgreSQL.